Twitter/X 媒体下载器

Java 重写版 详细设计文档

基于 twitter_download 项目功能进行开发

版本：V1.1　日期：2026-10-12

1. 设计概述

1.1 设计目标

本详细设计文档依据《Twitter/X 媒体下载器（Java 版）需求分析》编制，明确系统的整体架构、模块划分、核心类设计、数据模型、接口约定与关键流程，指导后续编码实现。设计目标是构建一套模块化、可扩展、易维护的本地媒体下载工具，完整覆盖用户媒体、Tag 搜索、评论区、纯文本、用户主页等下载能力。

1.2 设计原则

- 模块化：按职责拆分 Client、Task、Downloader、Storage 等核心模块，降低耦合。
- 可配置：GraphQL 查询定义、限流参数、命名模板等均可外部配置。
- 可扩展：任务类型通过统一接口抽象，新增模式只需实现对应 Task。
- 稳定性：完善的重试、限流、多账号切换与异常处理机制。
- 本地优先：所有数据与 Cookie 仅存储在本地，不依赖外部服务。

1.3 技术选型

模块	选型	说明
语言与运行时	Java 17+	长期支持版本，兼顾性能与生态
构建工具	Maven 或 Gradle	依赖管理与打包
HTTP 客户端	OkHttp 4.x	连接池、拦截器、代理、Cookie 管理
JSON 处理	Jackson	灵活解析 GraphQL 复杂响应
配置	SnakeYAML / Typesafe Config	YAML 配置，支持注释与环境变量
命令行	picocli	子命令、参数校验、帮助信息
数据库	SQLite + JDBC	本地轻量存储，记录下载状态

日志	SLF4J + Logback	分级日志与请求追踪
并发	CompletableFuture + ExecutorService	可控并发下载

2. 系统架构

2.1 整体架构

系统采用分层 + 模块化架构，自上而下分为：交互层、任务层、服务层、基础设施层。各层通过清晰接口通信，便于独立演进与测试。

【交互层】

CLI（picocli）负责参数解析、子命令分发、进度与结果输出。

【任务层】

统一 Task 抽象，具体实现包括 UserMediaTask、TagSearchTask、ReplyTask、TextTask、ProfileTask。

TaskScheduler 负责任务调度、并发控制与生命周期管理。

【服务层】

GraphQLClient：封装请求、认证头、Transaction-ID、限流与重试。

MediaDownloader：负责媒体文件下载、命名、落盘与重试。

AccountPool：多账号 Cookie 管理与轮换。

RateLimiter：请求频率与并发控制。

【基础设施层】

ConfigManager：加载与校验配置。

Storage（SQLite）：下载记录与状态持久化。

日志与工具类。

2.2 模块职责一览

模块	主要职责
cli	命令行入口、子命令定义、参数绑定、帮助与进度输出
config	配置加载、校验、默认值合并、环境变量覆盖
client	GraphQL 请求封装、Header 构造、Transaction-ID 生成、代理与 Cookie 注入
account	多账号 Cookie 池、轮换策略、账号健康状态管理
ratelimit	请求间隔、并发信号量、指数退避、Rate Limit 响应处理、账号每日调用配额账本
task	任务抽象与具体实现（用户媒体、Tag、评论区、纯文本、主页）
downloader	媒体下载、文件命名、目录创建、失败重试、进度回调

storage	SQLite 访问、下载记录 CRUD、增量判断与水线位查询、缓存清理、单写队列管理
model	领域模型（User、Tweet、Media、DownloadRecord 等）
util	通用工具（时间、路径、哈希、模板渲染等）

3. 包结构与目录设计

推荐包结构如下（根包名可自定义，示例使用 com.xmedia.downloader）：

```
com.xmedia.downloader
├── Application.java                  // 主入口
├── cli/
│   ├── MainCommand.java
│   ├── UserCommand.java
│   ├── TagCommand.java
│   ├── ReplyCommand.java
│   ├── TextCommand.java
│   └── ProfileCommand.java
├── config/
│   ├── AppConfig.java
│   └── ConfigLoader.java
├── client/
│   ├── GraphQLClient.java
│   ├── TransactionIdGenerator.java
│   └── RequestInterceptor.java
├── account/
│   ├── Account.java
│   └── AccountPool.java
├── ratelimit/
│   └── RateLimiter.java
├── task/
│   ├── Task.java                     // 接口
│   ├── AbstractTask.java
│   ├── UserMediaTask.java
│   ├── TagSearchTask.java
│   ├── ReplyTask.java
│   ├── TextTask.java
│   ├── ProfileTask.java
│   └── TaskScheduler.java
├── downloader/
│   ├── MediaDownloader.java
│   └── NamingTemplate.java
├── storage/
│   ├── Database.java
│   └── DownloadRecordRepository.java
├── model/
│   ├── User.java / Tweet.java / MediaItem.java ...
└── util/
```

4. 核心模块详细设计

4.1 配置模块（config）

4.1.1 配置结构

配置文件采用 YAML 格式，主要节点如下：

```
save-path: "./downloads"
db-path: "./data/xmedia.db"            # SQLite 数据库文件路径
log:
  level: INFO                          # ERROR / WARN / INFO / DEBUG
  file: "./logs/xmedia.log"
  max-history: 7                       # 日志轮转保留天数
accounts:
  - name: account1
    note: "本人账号（likes / 亮点 任务必须绑定）"
    cookie: "auth_token=xxx; ct0=yyy;"
proxy:
  mode: single                         # single | pool（pool 为预留，第二阶段实现）
  list:
    - "http://127.0.0.1:7890"
download:
  max-concurrent: 8
  has-video: true
  has-retweet: false
  time-range: "1990-01-01:2030-01-01"
  naming-template: "{user}/{date}_{tweetId}_{mediaKey}.{ext}"
  enable-incremental: true
  video-variant: highest               # highest | all | lowest
  resume-enabled: true                 # 断点续传：*.part 临时文件 + 校验后原子改名
rate-limit:
  request-interval-ms: 500
  max-retries: 3
quota:
  per-account-daily: 1000              # 账号每日 GraphQL 调用上限（按实测校准）
  reserve-for-retry: 50                # 为失败重试预留的调用数
  estimate-tweets-per-call: 19         # 启动前估算：调用次数 ≈ 推文数 / 19
```

配置优先级：命令行参数 > 环境变量（如 XMEDIA_*）> 配置文件 > 内置默认值。

启动前校验：Cookie 必填且格式合法、路径可写、并发数与间隔为正整数；并按 quota 估算本次所需调用数（目标推文数 ÷ estimate-tweets-per-call），若超出可用账号的剩余日配额，则拒绝启动并提示所需账号数量。

命名模板可用变量：{user} {date} {tweetId} {mediaKey} {type} {index} {ext}。其中 {index} 仅表示本轮运行内序号，重跑会变化；需要持久稳定的文件名请使用 {tweetId} 与 {mediaKey}。文件名落盘前统一经 PathSanitizer 清洗（非法字符替换、长度截断、Windows 保留名规避）。

4.1.2 关键职责

- ConfigLoader：读取 YAML，合并默认值与环境变量，执行必填项与格式校验。
- AppConfig：强类型配置对象，供各模块注入使用。

4.2 GraphQL 客户端（client）

4.2.1 GraphQLClient

职责：统一发起 GraphQL 请求，组装完整请求头、解析响应与基础错误。请求头不完整时，即使 Cookie 有效也会被拒，下表为必须齐备的最小集合：

请求头	取值 / 来源	是否可配置

cookie	账号池中该账号的 auth_token + ct0	否（来自账号配置）
x-csrf-token	与 ct0 同值	否
x-client-transaction-id	TransactionIdGenerator 生成，按 operation 需要	否（算法可插拔）
authorization	平台公开的 Web Bearer Token	是（随前端版本更新）
x-twitter-auth-type	OAuth2Session	否
x-twitter-active-user	yes	否
x-twitter-client-language	语言代码（默认 en）	是
User-Agent	固定为近期主流浏览器 UA	是（需与 Bearer 版本匹配）
content-type / accept	application/json	否

核心方法示例：

```
  <T> T execute(String operationName, String queryId, Map<String,Object> variables, Class<T>
type)
  JsonNode executeRaw(String operationName, String queryId, Map<String,Object> variables)
```

4.2.2 TransactionIdGenerator

职责：生成 X-Client-Transaction-ID。实现可替换，便于在算法变更时快速适配。适用范围：仅 Tag / 评论区等受平台校验的 operation 需要该头，用户媒体时间线与纯文本获取通常不受影响；社区参考实现（iSarabjitDhiman/XClientTransaction）存在概率性失效，需持续跟进维护。

接口建议：String generate(String path, String method)；失败处理：有限次重试（建议 2 次）内重新生成，仍失败则抛出明确异常并提示「Transaction-ID 算法可能已变更」。

4.2.3 请求拦截与重试

通过 OkHttp Interceptor 实现：注入账号 Cookie、统一 User-Agent、记录请求日志、捕获 429 / Rate Limit 响应并通知 RateLimiter 与 AccountPool。

重试策略：最多 N 次，指数退避，可切换账号后重试。

4.2.4 操作定义失效识别与恢复

• 错误白名单：将 GraphQL 业务错误中「操作不存在 / queryId 失效 / features 不匹配」类错误码与消息文本纳入白名单，命中即判定为操作定义失效。

• 明确提示：判定失效时抛出可读异常，提示用户更新 graphql-operations.yaml，而非笼统报「获取数据失败」。

• 配置自述：graphql-operations.yaml 顶部标注所用版本、抓取日期与来源说明，便于比对与回滚。

4.3 账号池（account）

• Account：封装 name、cookie、ct0、authToken、状态（可用/限流中/失效）、最后使用时间、当日已用调用数与剩余配额（quotaUsedToday / quotaRemaining）等。

• AccountPool：维护账号列表，提供 acquire() / acquire(String preferredName) / release() / markRateLimited() / markInvalid() / markQuotaExhausted() 等方法。

轮换策略：优先选择当前可用且最近未使用的账号；全部限流时进入等待或抛出友好异常；若任务声明 accountBinding 指定账号（Likes / 仅亮点 等「本人账号」语义），必须使用该账号，不得轮换。

4.4 限流器（ratelimit）

• 基于信号量控制最大并发请求数。

• 维护最小请求间隔（request-interval-ms）。

• 收到 Rate Limit 响应后，延长对应账号冷却时间，并触发全局短暂退避。

• 与 AccountPool 协同：限流账号自动切换到其他可用账号。

• 日调用配额：按（账号 × 自然日）累计 GraphQL 调用次数，落库于 quota_ledger 表；区别于频率限流，配额耗尽后退避与降并发均无效，只能换号或次日续跑。

• 配额耗尽处理：收到平台的每日配额耗尽响应时，标记账号（markQuotaExhausted）并抛 QuotaExhaustedException，禁止继续重试。

• 启动前估算：按「目标推文数 ÷ estimate-tweets-per-call」估算所需调用数，超出可用账号剩余配额时拒绝启动并提示所需账号数量。

4.5 任务体系（task）

4.5.1 Task 接口

```
public interface Task {
    String name();
    void execute(TaskContext context) throws Exception;
}
```

TaskContext 提供 GraphQLClient、MediaDownloader、Storage、Config、进度回调等依赖。

4.5.2 具体任务

任务类	职责摘要
UserMediaTask	按 screen_name 拉取用户媒体时间线，支持排除转推、亮点、喜欢、时间范围，解析媒体列表并提交下载；任务可声明 accountBinding（AUTO / 指定账号名），Likes 与仅亮点必须绑定本人账号
TagSearchTask	按 Tag 或高级搜索 filter 拉取结果，解析媒体与可选文本并下载
ReplyTask	根据用户或推文链接拉取评论区，提取媒体并下载
TextTask	拉取用户纯文本推文并保存为文本/报告
ProfileTask	获取用户头像、banner、简介等信息并保存

4.5.3 TaskScheduler

负责接收一个或多个 Task，按配置的并发度调度执行，汇总成功/失败结果，输出最终统计。

支持串行（多用户依次）或有限并行（多任务同时）模式。

4.6 下载器（downloader）

• MediaDownloader：接收 MediaItem 列表，使用线程池并发下载，调用 NamingTemplate 生成路径，写入文件；下载结果不直接写库，而是投递到 Storage 的单写队列异步落库（避免 SQLite 并发写锁）。

• NamingTemplate：根据模板字符串与上下文变量（user、date、tweetId、mediaKey、type、index、ext 等）渲染最终文件名与相对路径；默认模板使用 {tweetId}/{mediaKey} 等稳定键，{index} 仅为本轮运行内序号。

• 下载失败时按配置重试，下载支持 HTTP Range 续传与 *.part 临时文件（校验通过后原子改名）；仍失败则写入 FAILED 记录（下次运行允许重试覆盖，不视为「已下载」）并继续后续任务。

• 支持进度回调（已完成数 / 总数），供 CLI 刷新进度条或日志。

• 文件名与目录在落盘前经 PathSanitizer 清洗：非法字符替换、长度截断、Windows 保留名（CON / PRN / AUX 等）规避。

4.7 存储模块（storage）

4.7.1 数据表设计

表 download_record（核心）：

字段	类型	说明
id	INTEGER PK	自增主键
tweet_id	TEXT	推文 ID
media_url	TEXT	媒体原始 URL（展示用；唯一约束改用 tweet_id + media_key）
media_type	TEXT	image / video / gif
local_path	TEXT	本地相对或绝对路径
user_screen_name	TEXT	所属用户
task_type	TEXT	来源任务类型
status	TEXT	SUCCESS / FAILED（跳过仅计数，不落库）
downloaded_at	TEXT	下载完成时间

完整字段与索引以《数据库结构说明》为准（含 user_rest_id、media_key、variant_bitrate、attempt_count、sync_cursor、quota_ledger）。可选扩展表：user_profile（主页信息）、task_run（任务执行历史）等。

4.7.2 Repository 职责

• exists(tweetId, mediaKey)：仅当存在 status='SUCCESS' 记录时视为已下载；FAILED 记录不参与跳过。：判断是否已下载（增量依据）。

• save(record)：UPSERT 写入（ON CONFLICT(tweet_id, media_key) DO UPDATE），允许失败项重试覆盖。

• listFailed() / clearCache(user, mode)（mode：仅记录 / 记录+文件 / 含空目录）/ getCursor(user) / saveCursor(user, lastMaxTweetId) 等维护方法。

数据库文件默认放在用户目录或配置指定路径，程序启动时自动建表，并将遗留的 task_run.status = RUNNING 收敛为 ABORTED。连接参数：journal_mode = WAL、busy_timeout = 5000、synchronous = NORMAL；写入统一经单写线程队列串行化。

5. 关键流程设计

5.1 用户媒体下载主流程

1. CLI 解析参数与配置，构建 AppConfig、AccountPool、GraphQLClient、Storage 等。

2. 创建 UserMediaTask，传入目标 screen_name 列表与过滤条件。

3. TaskScheduler 执行任务：

   a. 调用 GraphQL 获取用户信息与媒体时间线（分页）。

   b. 解析推文与媒体列表，按时间范围与转推选项过滤。

   c. 对每条媒体查询 Storage，跳过已下载项（增量）：先按该用户的 sync_cursor 水位线提前终止分页，再对每条媒体按 (tweet_id, media_key) 查 status='SUCCESS' 记录跳过。

   d. 将待下载列表提交 MediaDownloader。

   e. 下载完成后写回 Storage，更新进度。

4. 全部完成后输出统计（调用次数、成功数、失败数、耗时）。

5. 异常路径：频率限流 → 切换账号或退避后重试；账号每日调用配额耗尽 → 优雅停机并写入续跑点（次日续跑）；Cookie 失效 → 标记账号并提示用户。

5.2 Tag / 高级搜索流程

与用户媒体流程类似，差异在于：

• 使用搜索类 GraphQL 操作，variables 中传入 tag 或 filter 字符串。

• 结果可能跨用户，命名模板中需体现 tag 或搜索条件。

• 可选同时落盘推文文本。

5.3 评论区与其他任务

ReplyTask：根据推文 ID 或用户时间线中的回复关系拉取评论，再走统一下载链路。

TextTask / ProfileTask：不经过 MediaDownloader，直接将文本或头像等写入文件系统与可选数据库表。

所有任务共享同一套 Client、AccountPool、RateLimiter 与 Storage，保证行为一致。

6. GraphQL 操作约定

为便于应对接口变更，将 operationName、queryId、features 等定义放在外部配置文件（如 graphql-operations.yaml）中，由 Client 加载。主要操作包括但不限于：

操作用途	说明
UserByScreenName	根据 screen_name 获取用户基本信息与 rest_id
UserMedia / UserTweets	用户媒体或推文时间线（分页）
SearchTimeline	Tag 或高级搜索结果
TweetDetail / Replies	单条推文详情与评论区
UserHighlights / Likes	亮点与喜欢时间线（在权限允许时）

实际 queryId 与 features 需与当前 X 前端保持同步，建议在文档或配置中集中维护，并提供更新说明；失效识别与提示流程见 4.2.4。

7. 异常处理与日志

7.1 异常分类

- ConfigException：配置缺失或格式错误，启动阶段直接失败并给出明确提示。
- AuthException：Cookie 失效或认证失败，标记账号并提示更新。
- RateLimitException：触发限流，进入退避与账号切换流程。
- QuotaExhaustedException：账号每日调用配额耗尽，停止重试并记录续跑点（次日续跑）。
- NetworkException：网络或代理问题，可重试。
- DownloadException：单文件下载失败，记录后继续其他文件。

7.2 日志规范

- 使用 SLF4J，级别：ERROR / WARN / INFO / DEBUG。
- 关键请求带简要上下文（账号名、operation、耗时）。
- 敏感信息（完整 Cookie）不得输出到日志。
- 最终汇总以 INFO 输出，失败明细可写独立文件或 DEBUG。

8. 部署与运行设计

• 使用 Maven/Gradle 打包为可执行 fat-jar（含依赖）：SQLite 驱动使用 sqlite-jdbc（无需系统安装 sqlite3），Windows / Linux / macOS 三平台各执行一次冒烟验证。

• 运行方式示例：

```
java -jar x-media-downloader.jar user --users elonmusk,openai --config config.yaml
java -jar x-media-downloader.jar tag --tag "#example" --config config.yaml
```

• 配置文件、数据库文件、下载目录均可通过参数或配置指定，默认相对当前工作目录。

• 支持 --dry-run：仅加载并校验配置、估算所需调用数与可用配额，不发起真实请求。

• 文档需包含：环境要求（Java 17+）、Cookie 获取步骤、配置示例、常见问题与限流说明。

9. 实现优先级建议

与需求分析中的阶段对应：

第一阶段：Config + GraphQLClient（基础，含完整请求头与 TxId 按需生成）+ AccountPool（多账号 Cookie 池）+ RateLimiter（含日配额账本）+ UserMediaTask + MediaDownloader（含断点续传）+ Storage（WAL + 单写队列 + 水位线）+ 基础 CLI + --dry-run。

第二阶段：代理（含代理池）、TagSearchTask、ReplyTask、TextTask、ProfileTask、进度与报告、accountBinding 完善。

第三阶段：TransactionId 强化、配置校验完善、统计报告增强、打包与完整文档、可选稳定性打磨。

10. 文档说明

本详细设计文档依据需求分析确定的功能范围编写，给出了系统架构、模块职责、核心类与数据模型、关键流程及实现优先级，可直接作为开发任务拆分与编码实现的依据。具体 GraphQL queryId 与 features 需在实现时根据当时 X 前端接口进行填充与维护。如需调整模块边界或补充接口细节，可在本设计基础上迭代。

11. 文档效力与修订记录

文档效力顺序：本套文档共五份（需求分析、详细设计、类图说明、时序图与关键流程图、数据库结构说明）。效力顺序为：需求分析 > 详细设计 > 类图说明 / 时序图与关键流程图 / 数据库结构说明（三者同级，冲突时以详细设计为准）。出现冲突时，以顺序在前的文档为准。

修订记录：

版本	日期	说明	备注
V1.1	2026-10-12	依据评审意见整改：H2 引入日调用配额模型与 QuotaExhaustedException（§4.1.1、§4.4、§7.1）；H3 增单写	评审后首版

		队列与 SQLite PRAGMA 约定（§4.6、§4.7.2）；H4 第一阶段纳入多账号池（§9）；H5/S6 稳定键改 rest_id / media_key（§4.7.1、§4.7.2）；H6 引入 accountBinding（§4.3、§4.5.2）；S1 代理支持 single/pool（§4.1.1）；S2 配置全量字段（§4.1.1）；S3 状态去掉 SKIPPED（§4.7.1）；S4 断点续传定义（§4.6）；S5 水位线增量（§5.1）；S9 命名模板稳定键（§4.1.1、§4.6）；S10 新增 4.2.4 失效识别（§6）；S11 补请求头表（§4.2.1）；S12 配额与限流区分（§4.4）；S13 收敛 TxId 适用范围（§4.2.2）；S14 增效力顺序与本表；P1 统一 operationName；P2 PathSanitizer；P4 打包细节；P6 依赖装配下沉 AppBootstrap；P7 --dry-run 与单测约定	

— 文档结束 —